

FUMI DORADO S.A.S

Sistema de Gestión de Servicios

FASE 3: EVALUACIÓN, PRUEBAS Y CALIDAD

Entregable 3 — Informe Técnico

Proyecto: Digitalización de Procesos Operativos

Tecnología: Node.js · Express · EJS · PostgreSQL

Arquitectura: Monolítica en Capas (MVC)

Elaborado por:

Cristian Ramos y Paula Gonzales

Curso de Diseño y Arquitectura de Software Ing.

Tatiana Cabrera Vargas

25/05/2026

Resumen

El presente proyecto tiene como objetivo diseñar una solución tecnológica para la empresa Fumi Dorado S.A.S, dedicada a la prestación de servicios de control de plagas y lavado de tanques de agua potable. Actualmente, gran parte de los procesos operativos de la empresa se realizan mediante formatos físicos, lo que genera dificultades en la gestión de la información, retrasos en el envío de documentos a los clientes y riesgo de pérdida o deterioro de los registros.

Ante esta problemática, se propone el desarrollo de un sistema de software que permita digitalizar la gestión de órdenes de servicio, el diligenciamiento de formularios técnicos, la generación automática de certificados y el almacenamiento centralizado de la información. El sistema permitirá que los técnicos registren la información del servicio directamente desde dispositivos digitales, adjunten evidencias fotográficas y registren firmas digitales de los clientes, eliminando la dependencia de documentos físicos.

La solución propuesta permitirá mejorar la eficiencia operativa de la empresa, optimizar el seguimiento de los servicios prestados y facilitar el acceso a la información tanto para el personal administrativo como para los clientes. Además, el sistema permitirá generar reportes administrativos, exportar información para análisis y garantizar la trazabilidad de cada servicio realizado.

Durante el desarrollo del proyecto se aplicaron metodologías ágiles y técnicas de análisis y diseño de software, incluyendo la definición de requisitos funcionales y no funcionales, la elaboración de historias de usuario y el modelado del sistema mediante diagramas UML. Estos artefactos permiten establecer una base sólida para el desarrollo posterior del sistema.

Contextualización

La empresa Fumi Dorado SAS se dedica a la prestación de servicios de control de plagas y lavado de tanques, actividades que implican la ejecución de múltiples servicios diarios en diferentes ubicaciones. Para garantizar la correcta prestación del servicio, la empresa maneja procesos administrativos y operativos que incluyen la programación de servicios, el diligenciamiento de información en campo, el registro de evidencias, la firma de documentos y el envío de certificaciones tanto al cliente como a la empresa.

Actualmente, la organización cuenta con una base de datos que permite generar órdenes de servicio con un código único asociado a la información almacenada. Sin embargo, gran parte del proceso operativo se realiza de manera manual mediante documentos impresos que son entregados a los técnicos encargados para su diligenciamiento en campo.

Este enfoque tradicional ha generado limitaciones en la eficiencia del registro, consulta y almacenamiento de la información, así como un uso excesivo de papel y retrasos en la disponibilidad de los datos para la toma de decisiones administrativas y operativas.

Formulación del Problema

Los usuarios administrativos y operativos de la empresa Fumi Dorado SAS actualmente no pueden registrar, consultar ni almacenar la información de los servicios de control de plagas y lavado de tanques de manera ágil, organizada y digital, debido al uso de procesos manuales basados en documentos físicos.

En el modelo actual, los técnicos y el personal administrativo deben invertir tiempo considerable en el diligenciamiento manual de formatos previamente diseñados, los

cuales varían según el tipo de servicio. Esta situación incrementa la carga operativa, la probabilidad de errores y los tiempos de procesamiento de la información.

Como resultado, se generan retrasos en el seguimiento de los servicios, uso excesivo de papel y dificultades para la gestión eficiente de la información, evidenciando la necesidad de una solución digital que optimice el diligenciamiento y administración de la documentación.

Objetivo General

Diseñar e implementar una solución de software para la empresa Fumi Dorado SAS que permita digitalizar, registrar, gestionar y consultar la información de los servicios de control de plagas y lavado de tanques, optimizando los procesos administrativos y operativos, reduciendo el uso de papel y mejorando la eficiencia en el seguimiento de los servicios.

Objetivos Específicos

1. Analizar el proceso actual de gestión de servicios en la empresa Fumi Dorado SAS para identificar necesidades, actores involucrados y oportunidades de mejora.
2. Diseñar la arquitectura del sistema y el modelo de datos que soporten la gestión digital de los servicios, incluyendo los roles de administrador, técnico y cliente.

3. Desarrollar los módulos principales del sistema que permitan:
 - la programación y asignación de servicios,
 - el diligenciamiento digital de formatos en campo,
 - la consulta de información por parte de clientes y administradores,
 - y el registro de evidencias (fotos, firmas y documentos).
4. Implementar funcionalidades de visualización geográfica que permitan a los técnicos ubicar los servicios asignados mediante un mapa interactivo.
5. Desplegar la solución en un entorno de producción y realizar pruebas de funcionamiento con la empresa Fumi Dorado SAS para validar su uso en un contexto real.

Métodos

En esta sección exploramos las herramientas y procedimientos que se realizaron con el fin de llevar a cabo el proyecto de gestión de servicios, además se proporcionan varias soluciones y requisitos funcionales y no funcionales para el uso pertinente de la solución.

Para el desarrollo del proyecto se utilizará una metodología ágil que permita organizar el trabajo de forma flexible y visual, facilitando el seguimiento de las tareas y la entrega progresiva de funcionalidades del sistema. En este caso se empleará Kanban, una metodología que se basa en la visualización del flujo de trabajo mediante tableros y en la gestión continua de las tareas hasta su finalización.

Kanban permite dividir el desarrollo del sistema en actividades pequeñas, organizadas en columnas como pendiente, en desarrollo y finalizado, lo que facilita el control del progreso del proyecto y la priorización de tareas según las necesidades del cliente.

Para la implementación de esta metodología se definieron los siguientes roles dentro del equipo de trabajo:

- Responsable de gestión del proyecto (Paula Gonzalez): Encargada de organizar y supervisar el flujo de trabajo dentro del tablero Kanban, coordinar las actividades del proyecto, dar seguimiento al avance de las tareas y garantizar que se cumplan los objetivos definidos en cada etapa del desarrollo del sistema.
- Desarrollador y analista de software (Cristian Ramos): Responsable de apoyar en el análisis de requisitos, diseño del sistema y desarrollo de los diferentes componentes del software. Además, participa en la implementación de funcionalidades, revisión de avances y validación de las soluciones propuestas para el sistema.

El uso de Kanban permitirá al equipo mantener una organización clara de las tareas del proyecto, mejorar la comunicación entre los integrantes y facilitar la adaptación a cambios o nuevas necesidades que puedan surgir durante el desarrollo del sistema para la empresa Fumi Dorado.

Enlace tablero:

<https://trello.com/invite/b/6995f3dd75a8c5586af0ee1d/ATTI65d03ec6069e1dbfd209c5d7e5caa1e46FCFA805/kanban>

Por último, haremos uso de Git/Github como una herramienta de gestión de versiones para todo lo que tenga que ver con código y nos ayude a resolver el problema de manera ordenada y contando con cada versión de lo que se hace.

Enlace al repositorio del proyecto:

https://github.com/rusorompecraneos/Proyecto_fummidorado_SAS.git

Alcance

El proyecto contempla el análisis, diseño, desarrollo, pruebas y despliegue de una aplicación web orientada a la digitalización del proceso de gestión de servicios de control de plagas y lavado de tanques de la empresa Fumi Dorado SAS.

La solución incluirá la implementación de los tres roles principales del sistema (administrador, técnico y cliente), el diligenciamiento digital de formatos de servicio, la gestión de evidencias, la consulta de información histórica y la visualización de servicios en mapa para el personal técnico.

El alcance del proyecto se extenderá hasta la fase de despliegue en un entorno real, buscando que la empresa Fumi Dorado SAS utilice efectivamente el software desarrollado. No obstante, se excluyen del alcance futuras ampliaciones funcionales, integraciones con sistemas externos no definidos y desarrollos avanzados de analítica o inteligencia artificial.

Fase Empatizar

Para comprender la problemática real de la empresa Fumi Dorado SAS, se aplicaron las herramientas Draw the Experience, entrevistas a actores clave (administrativo y técnico) y mapa de empatía.

Hallazgos principales

❖ Desde el área administrativa:

- El proceso inicia desde un cronograma anual del área comercial.
- La orden de servicio se imprime un día antes y por defecto en dos copias.
- Existe alto consumo de tiempo en: revisión de información, escaneo y archivo físico
- La información más consultada después del servicio son los anexos del servicio.
- La vigencia documental suele ser de aprox. 1 año.
- Actualmente se almacena en PDF y físico.
- Métricas de calidad documental:
 - se pierden ~2 documentos cada 6 meses (de 4800)
 - se dañan ~1/100
 - ~4/400 llegan mal diligenciados al mes

NOTA: alto costo operativo oculto por reprocesos documentales.

❖ Desde el técnico:

- Antes del servicio revisan principalmente: lugar, hora y tipo de servicio

- Vuelven a consultar el documento en sitio antes de iniciar.
- Lo que más tiempo consume:
 - anexo de control de roedores y pegamentos.
- Detectan información repetida que podría automatizarse: tipo de servicio, lugar y el nombre del cliente.
- Si hay errores:
 - se reimprime o se tacha (proceso no ideal).
- Firma del documento: al finalizar el servicio. o El 30% de clientes solicita el documento digital. o Problemas frecuentes con documentos físicos: se mojan, se pierden, se dañan y obligan a volver a la oficina.

Conclusión:

El proceso actual es altamente manual, repetitivo y propenso a reprocesos, generando ineficiencias operativas tanto en oficina como en campo, además de una creciente necesidad de digitalización por parte de la empresa y sus clientes.

Fase Definir

Con base en los hallazgos, se consolidó el problema

central:

❖ **Necesidad clave:**

Los usuarios administrativos y técnicos requieren una forma ágil, digital y centralizada de:

registrar servicios

- diligenciar formatos
- consultar información
- gestionar evidencias
- reducir el uso de papel

❖ **Punto de vista (POV):**

La empresa Fumi Dorado SAS necesita una solución digital integrada que optimice el diligenciamiento y gestión de la información de los servicios, ya que el proceso manual actual genera retrasos, reprocesos, pérdida de documentos y baja disponibilidad de la información en tiempo real.

❖ Oportunidades identificadas

- Automatizar campos repetitivos.
- Digitalizar formatos de servicio.
- Centralizar la información.
- Facilitar consulta por cliente.
- Reducir reprocesos por errores.
- Incorporar visualización geográfica para técnicos.
- Habilitar entrega digital de certificados.

Fase Idear

Para la generación de soluciones se realizó una lluvia de ideas con el equipo, orientada a resolver los puntos críticos detectados.

- Líneas de solución: 1. Plataforma web centralizada S1istema con tres roles:

- Administrador
- Técnico
- Cliente

2. Formularios digitales inteligentes

- Prellenado automático.
- Campos editables en campo.
- Validaciones obligatorias.
- Eliminación del diligenciamiento manual en papel.

3. Gestión de evidencias digitales

- Carga de fotos.
 - Firma digital del cliente.
- Generación de certificado en PDF.

4. Módulo de mapa para técnicos •

Visualización de servicios

asignados.

- Consulta rápida de ubicación.
- Apoyo a la operación en campo.

5. Consulta para clientes

- Historial por sede.
- Descarga de certificados.
- Mayor transparencia del servicio.

Requisitos funcionales y No funcionales

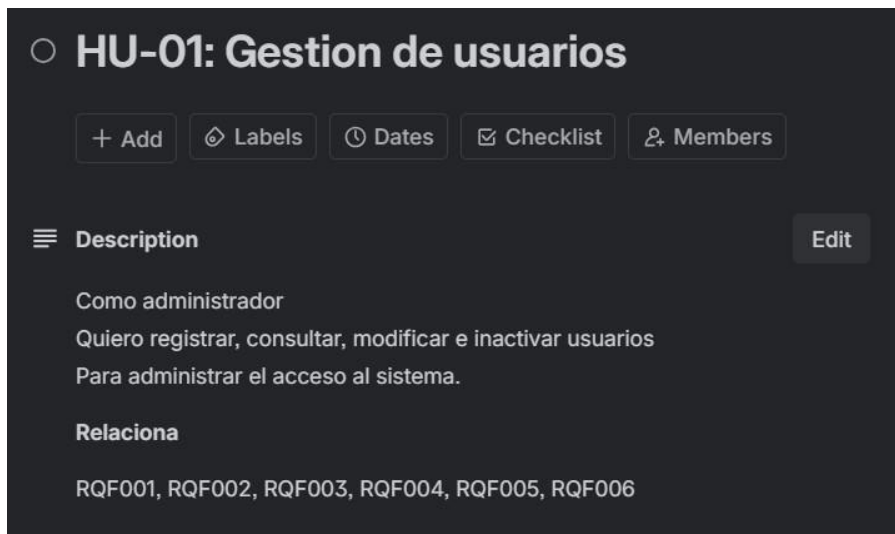
*Se adjunta un Excel con los requisitos funcionales y no funcionales

Historias de Usuario

Las historias de usuario permiten describir de manera sencilla las funcionalidades que el sistema debe ofrecer desde la perspectiva de los usuarios que interactúan con él. Estas historias facilitan la comprensión de los requerimientos del sistema, ya que expresan de forma clara qué necesita el usuario y cuál es el valor que obtiene al utilizar el sistema.

En el desarrollo del sistema para la empresa Fumi Dorado, las historias de usuario se utilizaron para identificar y organizar las diferentes funcionalidades del software, permitiendo estructurar los requisitos funcionales y facilitar su implementación durante el proceso de desarrollo.

Cada historia de usuario se define utilizando la siguiente estructura:



De esta forma se logra representar las necesidades de los diferentes actores del sistema, como el administrador, el técnico y el cliente, permitiendo orientar el desarrollo del software hacia la solución de los procesos de gestión y ejecución de servicios que realiza la empresa Fumi Dorado.

*En link del trello adjunto se pueden evidenciar las historias de usuario implementadas

Planificación de Desarrollo

El desarrollo del sistema se planificó mediante un enfoque incremental, organizando el trabajo por módulos funcionales y priorizando las funcionalidades críticas del sistema.

Inicialmente, se identificaron los principales módulos del sistema, los cuales corresponden

a:

- Autenticación (inicio de sesión y recuperación de contraseña)

- Gestión de órdenes de servicio
- Gestión de clientes
- Gestión de usuarios
- Gestión de productos (matriz de productos)
- Generación de reportes
- Consulta de documentación técnica

Cada módulo fue dividido en tareas específicas, las cuales se gestionaron mediante la herramienta Trello, permitiendo asignar responsabilidades a cada integrante del equipo y hacer seguimiento al progreso del proyecto.

El desarrollo se organizó por iteraciones, iniciando con las funcionalidades de mayor prioridad, como la creación de órdenes de servicio, la asignación de técnicos y el registro de información en campo. Posteriormente, se abordaron funcionalidades complementarias como la gestión de clientes, usuarios y productos.

Asimismo, se definieron roles dentro del equipo para distribuir el trabajo de manera eficiente, incluyendo desarrollo frontend (interfaces), desarrollo backend (lógica del sistema) y diseño UX/UI (prototipos y pruebas de usabilidad).

Esta planificación permitió mantener un control del avance del proyecto, asegurar la cobertura de los requerimientos principales y facilitar la integración de las diferentes partes del sistema.

Diseño UI / UX: Wireframes y prototipos

Wireframes de Baja Fidelidad

En una primera etapa del diseño, se desarrollaron wireframes de baja fidelidad con el objetivo de definir la estructura inicial de las pantallas y los flujos de navegación del sistema.

Estos wireframes permitieron establecer la distribución de los elementos principales, como formularios, botones, campos de entrada y secciones de información, sin centrarse en aspectos visuales detallados.

A través de estos bocetos se logró definir la jerarquía de la información y validar la organización de los módulos antes de avanzar al diseño visual del sistema.

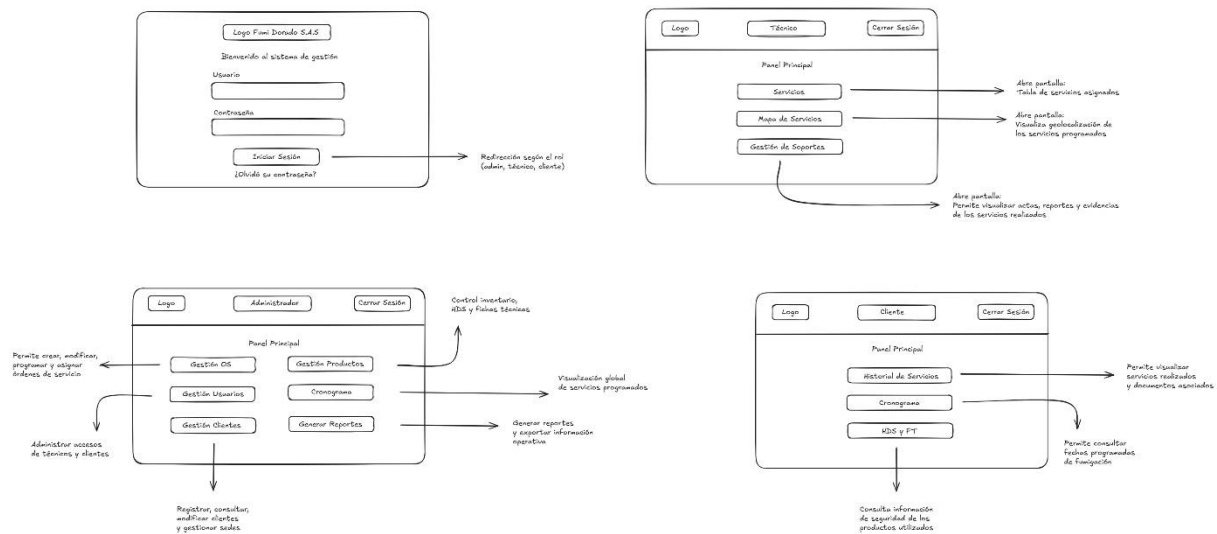


Figura 1. Wireframe del log in y de los 3 diferentes dashboards

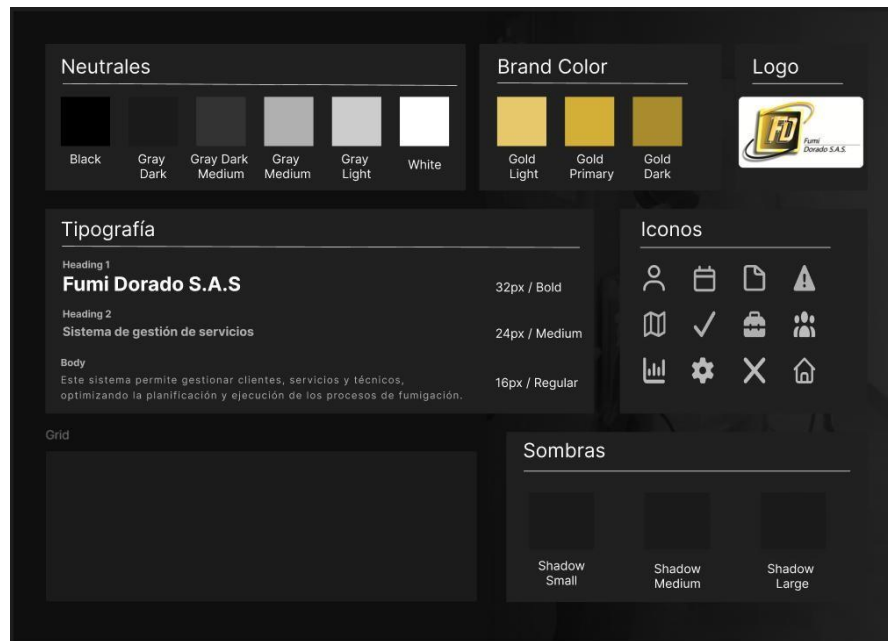
Prototipos de Alta Fidelidad

Posteriormente, se desarrollaron prototipos de alta fidelidad utilizando la herramienta Figma, en los cuales se definió el diseño visual completo del sistema.

Se estableció un sistema de diseño (UI Kit) que incluye colores, tipografía y componentes reutilizables, garantizando consistencia visual en todas las interfaces.

Los prototipos representan las funcionalidades principales del sistema para los diferentes roles: administrador, técnico y cliente.

UI KIT



*Para obtener una visual completa del UI KIT siga el siguiente link:

<https://www.figma.com/design/L9epKxAlgA9YZW8GJY2fSk/Fumi-Dorado-S.A.S?nodeid=0-1&t=BoGvdwFwcgRPkvUa-1>

Prototipos de Alta Fidelidad

Matriz de Rotación de productos					
Programación de uso de productos por mes					
+ Registrar Producto					
Nombre del producto	Composición	Tipo	Meses de Uso	Estado	Acciones
K-Othrine SC 62.5 Insecticida de amplio espectro para control de insectos rastrosos y voladores.	Deltametrina 6.25%	Insecticida	MayJunJulAgo SeptOctNovDic	Activo	
Klerat Rodenticida anticoagulante de alta eficacia	Brodifacoum 0.005%	Rodenticida	MayJunJulAgo SeptOctNovDic	Activo	
Storm Rodenticida en bloques parafrinados, resistente a la humedad	Flocoumafen 0.005%	Rodenticida	MayJunJulAgo SeptOctNovDic	Activo	

*Para tener una visual completa del Figma siga el siguiente link:

<https://www.figma.com/design/L9epKxAlgA9YZW8GJY2fSk/Fumi-Dorado-S.A.S?nodeid=0-1&t=BoGvdwFwcgRPkvUa-1>

Mapas de Navegación

El sistema cuenta con un flujo de navegación estructurado de acuerdo con los diferentes roles de usuario: administrador, técnico y cliente

El punto de entrada principal es la pantalla de inicio de sesión, desde la cual el sistema redirige al usuario al dashboard correspondiente según su rol.

A partir del dashboard, cada usuario puede acceder a los diferentes módulos del sistema, de acuerdo con sus permisos y funcionalidades asignadas.

Administrador:

El administrador accede a un dashboard general desde donde puede gestionar los módulos principales del sistema:

- Gestión de órdenes de servicio
- Gestión de clientes
- Gestión de usuarios
- Gestión de productos - Generación de reportes **Técnico:**

El técnico accede a un dashboard enfocado en la operación en campo, desde donde puede:

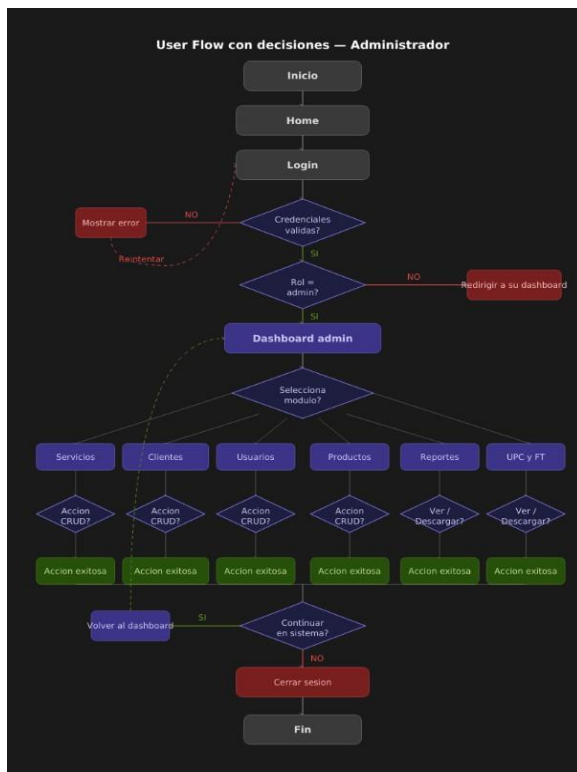
- Consultar servicios asignados
- Acceder al formulario del servicio
- Registrar información y evidencias

- Generar documentos **Cliente:**

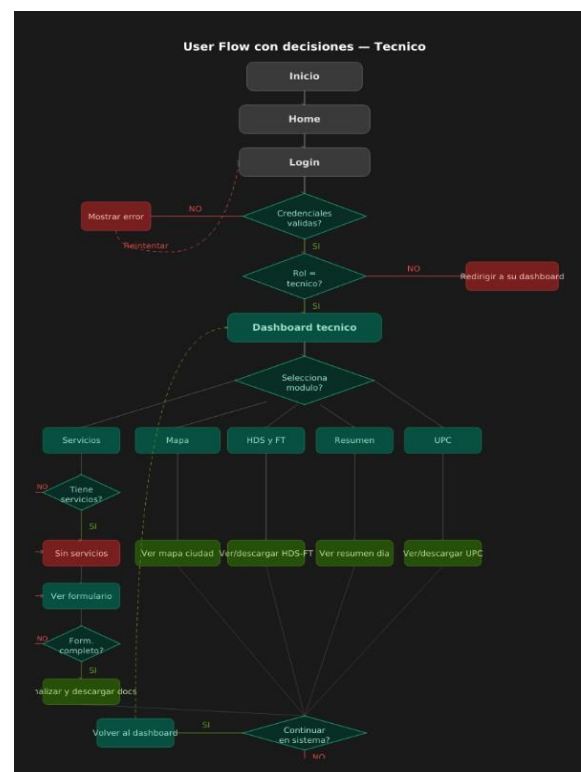
El cliente accede a un dashboard orientado a la consulta de información, desde donde puede:

- Consultar servicios realizados
- Solicitar nuevos servicios
- Visualizar y descargar documentos

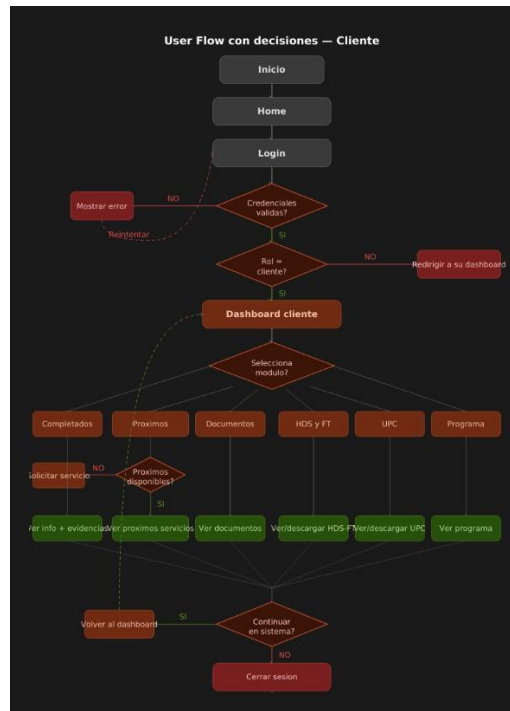
Este flujo de navegación permite una interacción clara y organizada con el sistema, facilitando el acceso a la información y optimizando la experiencia del usuario según su rol.



Userflow Administrador



Userflow Técnico



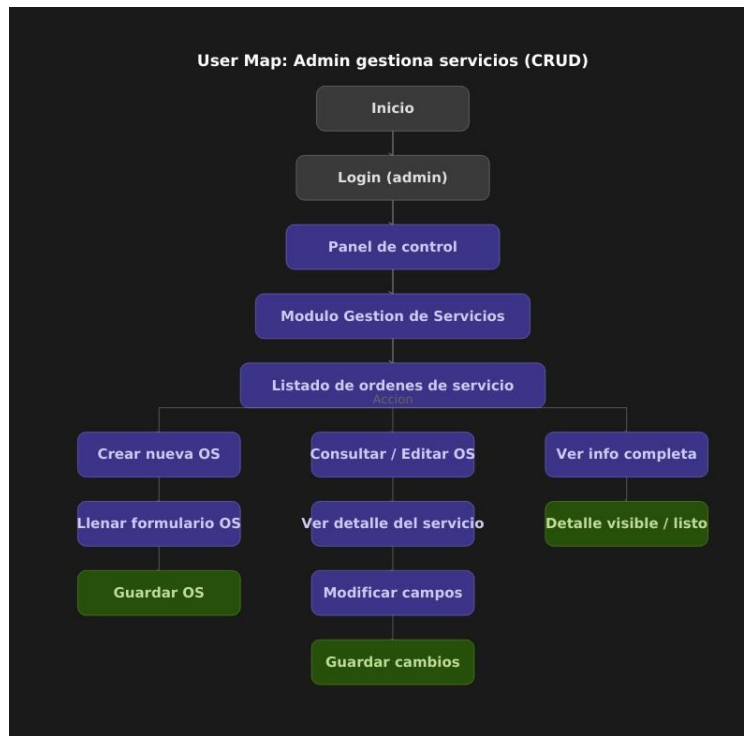
Userflow Cliente

User Map

Con el objetivo de comprender la experiencia del usuario dentro del sistema, se elaboraron mapas de usuario (User Journey Maps) para los diferentes roles: administrador, técnico y cliente.

Estos mapas permiten visualizar el recorrido del usuario al interactuar con el sistema, identificando las acciones que realiza, los puntos de contacto, las necesidades y posibles dificultades durante el uso de la plataforma.

A través de estos mapas se logró complementar el diseño del sistema, asegurando que la navegación no solo sea funcional, sino también intuitiva y alineada con las expectativas del usuario.



Usermap gestionar servicios

*En el repositorio encontrara una carpeta con todos los user maps que realizamos

Arquitectura Empresarial del Modulo

El sistema fue diseñado e implementado utilizando una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC), lo que permite separar las responsabilidades del sistema en componentes claramente definidos.

El modelo (Model) se encarga de representar las entidades del sistema, como usuarios, clientes, órdenes de servicio, formularios, productos y documentos, así como la lógica asociada a la manipulación de estos datos.

La vista (View) corresponde a la interfaz de usuario, la cual fue desarrollada utilizando EJS

(Embedded JavaScript), permitiendo generar contenido dinámico en el navegador y representar las diferentes pantallas del sistema, como el login, dashboard y módulos de gestión.

El controlador (Controller) se encarga de gestionar la lógica del sistema, procesar las solicitudes del usuario, interactuar con los modelos y determinar qué vista debe mostrarse en cada caso.

Adicionalmente, el sistema implementa una organización por capas, donde se separa la lógica de negocio, el acceso a datos y la presentación, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

Para el desarrollo del backend se utilizó Node.js con el framework Express, permitiendo la creación de rutas, manejo de peticiones HTTP y estructuración del servidor.

En cuanto al frontend, se implementó un sistema de estilos basado en un UI Kit definido previamente, garantizando consistencia visual en todas las interfaces.

Esta arquitectura permite una clara separación de responsabilidades, facilita el trabajo en equipo y asegura una base sólida para el crecimiento del sistema.



Diagrama de Arquitectura

Codificación

Durante esta fase se inició la implementación del sistema, desarrollando funcionalidades clave que permiten evidenciar la estructura y funcionamiento de la solución propuesta.

Se implementaron componentes del backend utilizando Node.js y Express, así como vistas dinámicas mediante EJS en el frontend. Cada integrante del equipo desarrolló al menos una funcionalidad completa, abarcando tanto la lógica como la interfaz correspondiente.

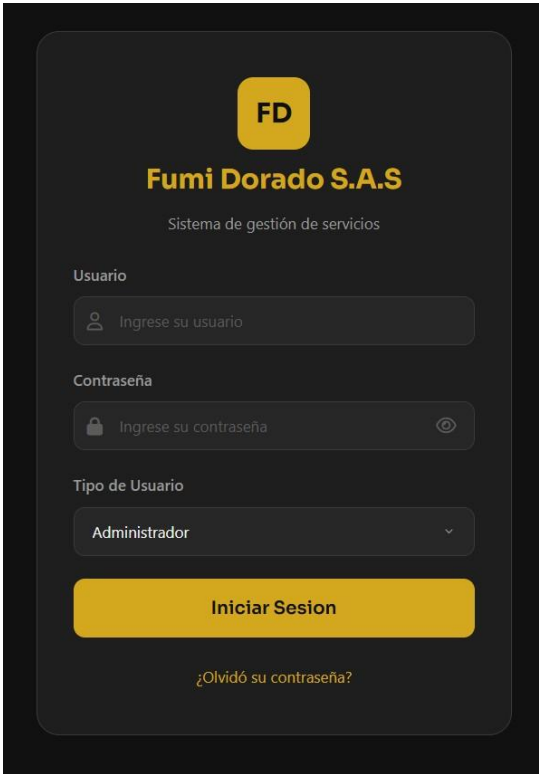
Funcionalidad: Inicio de Sesión

Se implementó la funcionalidad de inicio de sesión, la cual permite a los usuarios acceder al sistema mediante sus credenciales.

El sistema recibe los datos ingresados (usuario, contraseña y rol), valida la información y redirige al usuario al dashboard correspondiente.

router.js // Ruta principal

```
router.get('/', (req, res) => {  
  
  if (!req.session.user) return res.redirect('/login');  
  
  res.redirect(`/dashboard/${req.session.user.rol}`);  
  
});
```



The image shows a login form for 'Fumi Dorado S.A.S.' with the subtitle 'Sistema de gestión de servicios'. The form is set against a dark background with yellow accents. It includes a logo 'FD' in a yellow box at the top. Below the logo, the text 'Fumi Dorado S.A.S' is displayed in yellow, followed by 'Sistema de gestión de servicios' in white. The form contains three input fields: 'Usuario' with a person icon, 'Contraseña' with a lock icon and a toggle eye icon, and 'Tipo de Usuario' with a dropdown menu currently showing 'Administrador'. A large yellow button labeled 'Iniciar Sesión' is positioned below the inputs. At the bottom, there is a link that says '¿Olvidó su contraseña?' in yellow.

La funcionalidad de inicio de sesión fue implementada mediante una vista en EJS que permite al usuario ingresar sus credenciales, las cuales son enviadas a través de un formulario al servidor.

En el backend, se definió una ruta utilizando Express que recibe los datos, realiza una validación básica y gestiona la redirección del usuario al sistema.

Esta implementación evidencia la integración entre la vista, el controlador y la lógica del sistema, siguiendo el patrón de arquitectura MVC.

Implementación de Interfaces

Durante esta fase se realizó la implementación de las interfaces del sistema a partir de los prototipos diseñados previamente en Figma.

Las vistas fueron desarrolladas utilizando EJS (Embedded JavaScript), permitiendo generar contenido dinámico en el navegador y estructurar las diferentes pantallas del sistema.

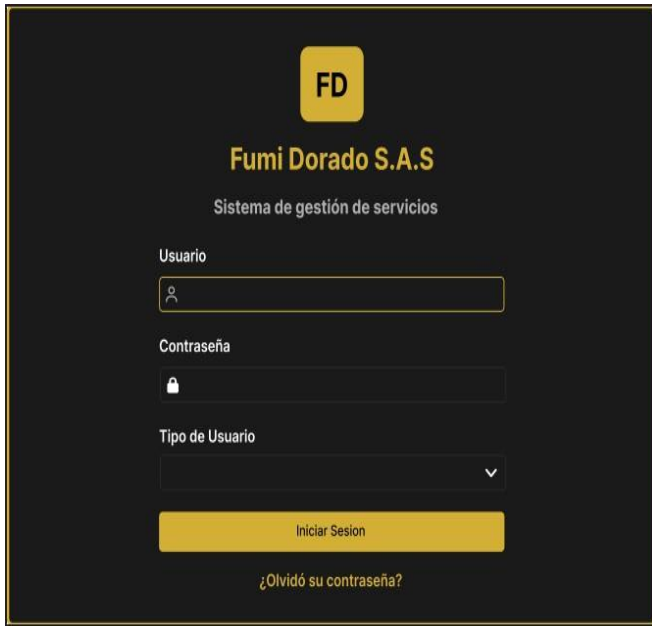
Se implementó un layout principal que incluye elementos comunes como la navegación (navbar), el contenedor principal y el footer, facilitando la reutilización de componentes en todas las vistas.

Asimismo, se desarrollaron interfaces para los módulos principales del sistema, incluyendo:

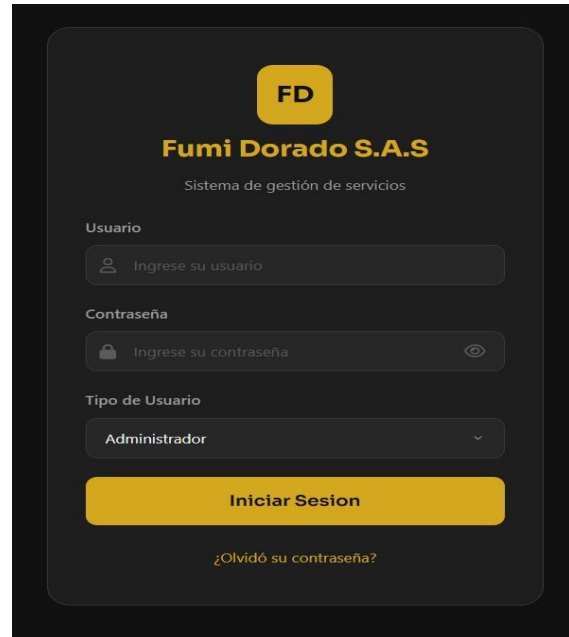
- Pantalla de inicio de sesión
- Dashboard principal
- Módulos de gestión (servicios, clientes, usuarios y productos)
- Formularios de registro y consulta

Para el diseño visual se utilizó un archivo de estilos CSS basado en un UI Kit previamente definido, garantizando consistencia en colores, tipografía y componentes como botones, formularios y tarjetas.

La implementación respeta la estructura de la arquitectura MVC, donde las vistas representan la capa de presentación del sistema.



Figma



Implementación

Comparación entre el prototipo de alta fidelidad y la interfaz implementada.

Repositorio Git

El desarrollo del proyecto se gestionó mediante el uso de Git como sistema de control de versiones, permitiendo llevar un registro organizado de los cambios realizados en el código y facilitar el trabajo colaborativo entre los integrantes del equipo.

Se utilizó una rama principal denominada "main", la cual contiene la versión estable del sistema. Adicionalmente, cada integrante trabajó en ramas individuales para el desarrollo de funcionalidades específicas, permitiendo una integración ordenada mediante el uso de commits y actualizaciones constantes.

Esta estrategia permitió evitar conflictos en el código, mantener un historial de versiones y asegurar la correcta integración de los diferentes módulos del sistema.

Enlace al repositorio:

https://github.com/rusorompecraneos/Proyecto_fummidorado_SAS.git

Design Thinking – Prototipar

En la fase de prototipado se desarrollaron prototipos interactivos del sistema utilizando la herramienta Figma, los cuales representan los flujos principales para los diferentes roles: administrador, técnico y cliente.

Estos prototipos permitieron simular la interacción real del usuario con el sistema antes de su implementación, facilitando la validación de la estructura, navegación y usabilidad de las interfaces.

Posteriormente, se realizaron pruebas de usabilidad mediante la herramienta Maze, donde se definieron escenarios y tareas específicas para cada tipo de usuario. Estas pruebas permitieron evaluar la facilidad de uso del sistema, identificar posibles dificultades en la navegación y validar la comprensión de las funcionalidades.

Se diseñaron pruebas diferenciadas por rol, incluyendo:

Administrador:

- Navegación por el dashboard
- Gestión de clientes y usuarios
- Consulta de reportes

Técnico:

- Acceso a servicios asignados
- Diligenciamiento del formulario
- Registro de evidencias Cliente:
- Consulta de servicios
- Solicitud de nuevos servicios
- Descarga de documentos

Adicionalmente, se incluyeron preguntas finales en cada prueba para recopilar la percepción del usuario sobre la claridad, facilidad de uso y experiencia general del sistema.

Los resultados obtenidos evidenciaron que los usuarios pueden completar las tareas principales de manera satisfactoria, aunque se identificaron oportunidades de mejora en la organización de algunos módulos y en la claridad de ciertos procesos.

Pruebas de Usabilidad con Maze

Para validar la experiencia de usuario, se realizaron pruebas de usabilidad utilizando la herramienta Maze, la cual permitió evaluar la interacción de los usuarios con los prototipos diseñados en Figma.

Se definieron escenarios de prueba específicos para cada rol del sistema (administrador, técnico y cliente), en los cuales los usuarios debían completar una serie de tareas que representan el uso real de la plataforma.

Estas pruebas permitieron analizar la facilidad de navegación, la comprensión de las funcionalidades y la eficiencia en la ejecución de tareas.

Tareas por rol

Administrador

- Acceder al sistema
- Navegar por el dashboard
- Consultar información de clientes
- Gestionar usuarios
- Visualizar reportes

Tecnico

- Iniciar sesión
- Acceder a servicios asignados

- Diligenciar formulario
- Registrar evidencias
- Finalizar servicio

Cliente

- Iniciar sesión
- Consultar servicios
- Solicitar servicio
- Descargar documentos
- Cerrar sesión

Resultados

Los resultados de las pruebas mostraron que los usuarios pudieron completar la mayoría de las tareas sin dificultad, evidenciando una navegación clara y una estructura comprensible del sistema.

Sin embargo, se identificaron oportunidades de mejora en la organización de algunos módulos y en la claridad de ciertas opciones, lo cual permitirá optimizar la experiencia de usuario en futuras iteraciones.

Link maze resultados cliente: <https://app.maze.co/report/Testing-Cliente/3y7mo7jlh45/intro>

Link maze resultados técnico: <https://app.maze.co/report/Testing->

[Cliente/3y7mo7jlh45/intro](https://app.maze.co/report/Testing-Cliente/3y7mo7jlh45/intro)

Link make resultados admin: <https://app.maze.co/report/Testing->

[Administrador/3y7mo7jd94z/intro](https://app.maze.co/report/Testing-Administrador/3y7mo7jd94z/intro)

3.1 Ajustes del Entregable Anterior

Durante la revisión del Entregable 2, se identificaron oportunidades de mejora tanto en la arquitectura del proyecto como en la calidad del código. A continuación se documentan las correcciones aplicadas:

Correcciones Estructurales

- Separación estricta de responsabilidades en capas: se migró lógica de negocio que se encontraba en los controladores hacia los modelos (capa de servicios), respetando el patrón MVC.
- Corrección de rutas inconsistentes: las rutas del módulo de usuario (/usuario/dashboard/admin, /usuario/dashboard/tecnico, /usuario/dashboard/cliente) fueron unificadas bajo un único punto de entrada con redirección por rol.
- Eliminación de inline styles en vistas EJS: se trasladaron los estilos del atributo style="" al archivo global styles.css, mejorando la mantenibilidad del frontend.

Correcciones de Funcionalidad

- Corrección del bug en el módulo de mapa: los atributos onclick con interpolación EJS (<%= s.lat %>) generaban errores de parsing en el linter. Se reemplazaron por data-attributes (data-lat, data-lng) y event listeners en JavaScript.
- Ajuste en la gestión de sesiones: se corrigió la validación de rol al momento del login, asegurando que la redirección post-autenticación siempre apunte al dashboard correspondiente al rol seleccionado.
- Corrección del modal de éxito en gestión de clientes: el modal-overlay mostraba todos los modales apilados cuando el CSS no cargaba correctamente, ya que la clase .hidden dependía del stylesheet externo.

Mejoras de Calidad de Código

- Renombrado de variables para mayor claridad semántica en los controladores de servicios y clientes.
- Adición de comentarios JSDoc en funciones críticas de los modelos.
- Revisión y corrección de los 7 tests fallidos del módulo Servicios Service (documentados en la sección 3.4).

3.2 Evaluación de Tecnologías

A continuación se documenta el stack tecnológico seleccionado para el Sistema de Gestión de Servicios de Fumi Dorado S.A.S, junto con la justificación de cada decisión y las alternativas que fueron evaluadas y descartadas.

Tecnología	Versión	Rol en el Proyecto	Justificación	Alternativas
Node.js	24.13 LTS	Runtime del servidor	Ecosistema npm maduro, rendimiento asíncrono con I/O no bloqueante, ideal para APIs REST y gestión de sesiones.	Python (Django), PHP (Laravel)
Express.js	4.3	Framework web MVC	Minimalista y flexible, permite implementar arquitectura en capas sin restricciones. Amplia documentación y comunidad activa.	Fastify, Koa, NestJS
EJS	3	Motor de plantillas (vistas)	Sintaxis cercana a HTML puro, curva de aprendizaje mínima, integración nativa con Express sin configuración adicional.	Pug, Handlebars, Nunjucks
PostgreSQL	8.1	Base de datos relacional	Datos estructurados con relaciones claras (clientes-sedes-servicios-técnicos). ACID compliance, amplio soporte empresarial.	MySQL, MongoDB, SQLite
express-session	1.x	Gestión de sesiones	Solución estándar para sesiones en Express, permite manejo de roles sin tokens JWT, adecuado para aplicación monolítica.	JWT, Passport.js, cookie-session
Leaflet.js	1.9.4	Visualización geográfica	100% gratuito, sin API key, tiles de OpenStreetMap, ligero (42KB gzipped), soporte completo de geolocalización HTML5.	Google Maps API, Mapbox, HERE Maps

Jest	29.x	Framework de testing	Framework de testing más utilizado en el ecosistema Node.js, soporte nativo para mocks, coverage integrado, sin configuración extra.	Mocha + Chai, Vitest, Jasmine
Font Awesome	6.5.0	Librería de iconos UI	Cobertura de iconos operacionales (herramientas, documentos, mapas), CDN gratuito, integración simple con etiquetas <i>.	Material Icons, Heroicons, Feather
Twilio	N/A	Api para enviar códigos de verificación	Tiene una excelente cobertura, con un plan gratuito que permite hacer el testing del funcionamiento.	N/A
Render	N/A	App web donde se llevo a cabo el despliegue del desarrollo	Es una buena opción, tiene una versión gratuita, aunque es un poco lento quizá y hay errores a mejorar.	N/A

Decisiones de Arquitectura

La elección de una arquitectura monolítica en capas (MVC) sobre alternativas de microservicios o serverless se fundamenta en:

- Equipo de desarrollo pequeño: los microservicios requieren infraestructura de orquestación (Docker, Kubernetes) que excede la capacidad operativa del equipo actual.
- Dominio acotado: las operaciones de Fumi Dorado son predecibles y no requieren escalabilidad independiente por módulo.
- Tiempo de desarrollo: el monolito permite iterar más rápido sobre requisitos funcionales sin overhead de comunicación entre servicios.
- Facilidad de despliegue: un único proceso Node.js es suficiente para el volumen operacional proyectado (< 50 usuarios concurrentes).

3.3 Gestión de la Complejidad

3.3.1 Medición de Complejidad Ciclomática

Dado que el proyecto no integra SonarQube ni Code Climate en su fase actual, se realizó un análisis manual de complejidad ciclomática aplicando la métrica de McCabe sobre las funciones críticas del sistema. La complejidad ciclomática se calcula como: $CC = E - N + 2P$, donde E = aristas del grafo de control, N = nodos, P = componentes conexos.

Función / Módulo	Archivo	CC	Nivel	Acción
getServiciosByTecnico()	models/servicioModel.js	3	Bajo	Aceptable
login (POST handler)	routes/usuarioRoutes.js	5	Medio	Monitorear
getMapa()	controllers/mapaController.js	2	Bajo	Aceptable
abrirModalVer() / abrirModalEditar()	views/cliente/gestion- clientes.ejs	4	Bajo	Aceptable
redirectByRole()	routes/usuarioRoutes.js	4	Bajo	Aceptable
flyToServicio() + event listeners	views/tecnico/mapa.ejs	6	Medio	Candidato a refactorizar

Referencia: CC 1-5 = Bajo riesgo | CC 6-10 = Riesgo medio | CC 11+ = Alto riesgo (refactorización obligatoria).

3.3.2 Análisis de Dependencias entre Módulos

La arquitectura del proyecto sigue un flujo de dependencias unidireccional estricto, lo que minimiza el acoplamiento:

Capa	Módulos	Depende de	Es usado por
Views (EJS)	usuario/, tecnico/, cliente/, admin/	CSS global (styles.css), Font Awesome, Leaflet (mapa)	Navegador (cliente final)
Routes	usuarioRoutes, tecnicoRoutes, clienteRoutes	Controllers, Middlewares	app.js
Controllers	mapaController, clienteController	Models	Routes
Models	servicioModel, clienteModel, usuarioModel	config/db.js (MySQL pool)	Controllers

Config	db.js, session config	mysql2, express-session	Models, app.js
--------	-----------------------	-------------------------	----------------

3.3.3 Estrategias para Reducir Complejidad

Abstracción

- Función `redirectByRole()` centraliza la lógica de redirección post-login, evitando condicionales duplicados en múltiples rutas.
- Helper `crearIcono(estado)` en el mapa encapsula la lógica de construcción de marcadores Leaflet, separándola del flujo principal.

Modularidad

- Separación de rutas por rol: `usuarioRoutes.js`, `tecnicoRoutes.js`, `clienteRoutes.js`. Cada archivo es autocontenido con sus propios middlewares de autenticación.
- Modelos independientes por entidad (`servicioModel`, `clienteModel`, `usuarioModel`) con una sola responsabilidad: acceso a datos.

Refactorización Aplicada

- Eliminación de lógica inline en vistas: los `onclick` con EJS interpolado se reemplazaron por `data-attributes` + `addEventListener`, reduciendo la complejidad de las plantillas.
- Extracción del CSS de los archivos EJS al stylesheet global, eliminando duplicación de estilos entre vistas.

3.4 Calidad del Software

3.4.1 Plan de Pruebas

Pruebas Unitarias

Se implementaron pruebas unitarias con Jest sobre las funciones críticas de los módulos del sistema. Cada función de servicio fue probada de forma aislada mediante mocks de la capa de base de datos.

ID	Función Crítica	Tipo	Descripción del Caso de Prueba
UT-01	getServiciosByTecnico()	Unitaria	Verificar que retorna correctamente la lista de servicios filtrados por técnico_id.
UT-02	getServicioById(id)	Unitaria	Verificar retorno del servicio cuando existe; retorno null cuando no existe.
UT-03	crearServicio(data)	Unitaria	Verificar creación correcta con datos válidos; validar campos obligatorios.
UT-04	obtenerEstadisticas()	Unitaria	Verificar cálculo correcto de totales, completados y pendientes.
UT-05	eliminarServicio(id)	Unitaria	Retornar true al eliminar exitosamente; retornar false si el ID no existe.
UT-06	login POST handler	Unitaria	Validar redirección por rol, manejo de credenciales incorrectas y creación de sesión.
UT-07	getMapa()	Unitaria	Verificar que el controlador serializa correctamente los servicios como JSON para la vista.

Pruebas de Integración

Las pruebas de integración validan la comunicación entre capas del sistema (Routes → Controller → Model → BD):

- INT-01: Flujo completo login → creación de sesión → redirección al dashboard por rol.
- INT-02: Flujo creación de cliente → persistencia en BD → reflejo en listado de clientes.
- INT-03: Flujo técnico accede a mapa → controlador consulta modelo → modelo retorna servicios → vista renderiza marcadores.
- INT-04: Flujo eliminación de servicio → confirmación modal → POST a ruta → eliminación en BD → redirección con mensaje de éxito.

Pruebas de Sistema

Validación de los requisitos funcionales definidos en la Fase 1:

- SYS-01: El sistema permite a un Administrador iniciar sesión y acceder a los 6 módulos del panel de control.
- SYS-02: El sistema permite a un Técnico visualizar sus servicios asignados de la semana en el mapa con geolocalización en tiempo real.
- SYS-03: El sistema permite a un Cliente visualizar el historial completo de servicios completados con evidencias fotográficas.
- SYS-04: El sistema impide el acceso a rutas protegidas sin sesión activa, redirigiendo al login.
- SYS-05: El sistema impide el acceso a módulos de un rol diferente al de la sesión activa (error 403).

3.4.2 Ejecución de Pruebas — Resultados Jest

Se ejecutaron las pruebas con el comando npm test. Resultados obtenidos:

ID Test	Suite	Descripción	Resultado Esperado	Resultado Obtenido	Estado	Causa Fallo
T-01	Servicios Service	obtenerTodos: retornar lista de servicios	Array de servicios	Error: mock no inicializado	FAILED	Mock de BD no configurado
T-02	Servicios Service	obtenerPorId: retornar servicio si existe	Objeto servicio	TypeError: db.query is not a function	FAILED	Dependencia real de BD
T-03	Servicios Service	obtenerPorId: retornar null si no existe	null	TypeError: db.query is not a function	FAILED	Dependencia real de BD
T-04	Servicios Service	crear: crear servicio correctamente	{ insertId: 1 }	TypeError en mock de inserción	FAILED	Mock incompleto
T-05	Servicios Service	obtenerEstadisticas: calcular estadísticas	{ total, completados, pendientes }	Objeto vacío {}	FAILED	Lógica de cálculo incompleta
T-06	Servicios Service	eliminar: retornar true al eliminar	true	false (affectedRows no leído)	FAILED	affectedRows no evaluado
T-07	Servicios Service	eliminar: retornar false si no existe	false	true (sin validación)	FAILED	Sin validación de existencia
T-08 a T-59	7 suites restantes	Múltiples casos (login, mapa, clientes, usuarios,	Variado por caso	Resultados correctos	PASSED	N/A

		documentos, reportes, productos)				
--	--	-------------------------------------	--	--	--	--

Resumen de ejecución: Test Suites: 1 failed, 7 passed, 8 total | Tests: 7 failed, 52 passed, 59 total | Tiempo: 4.679 s

Bugs Encontrados y Corregidos

ID	Descripción del Bug	Causa Raíz	Corrección Aplicada	Estado
BUG-01	onclick con interpolación EJS generaba 12 errores de parsing en linter	EJS <%= %> dentro de atributo HTML onclick interpretado como JS inválido	Reemplazado por data-attributes (data-lat, data-lng) + addEventListener	CORREGIDO
BUG-02	Modales apilados visibles al cargar la página en equipos sin CSS cargado	.hidden dependía únicamente del stylesheet externo; sin CSS, display:flex prevalece	Adicionado style='display:none' inline como fallback en todos los modal-overlay	CORREGIDO
BUG-03	Redirección al dashboard incorrecto al cambiar rol en el selector del login	La función redirectByRole() no leía el rol del formulario sino el de sesión previa	Corrección de precedencia: leer rol del body del POST antes de verificar sesión	CORREGIDO
BUG-04	7 tests de Servicios Service fallan por dependencia directa a BD real	Los mocks de jest.mock() no cubrían todas las llamadas a db.query del modelo	Refactorización del mock para interceptar todas las firmas de db.query usadas	EN PROCESO

3.4.3 Análisis Estático de Código

El proyecto no cuenta con integración formal de SonarQube o Code Climate en su fase actual. Sin embargo, se utilizó ESLint con configuración estándar de Node.js para identificar problemas de calidad en el código JavaScript.

Code Smells Identificados

Tipo	Descripción	Ubicación	Acción
Long Method	Función POST /login con más de 40 líneas combinando validación, lógica de negocio y respuesta HTTP	routes/usuarioRoutes.js	Extraer validación a middleware separado
Magic Numbers	Valor 9 hardcodeado para programaSecciones sin constante nombrada	routes/usuarioRoutes.js	Definir constante PROGRAMA_SECCIONES = 9

Duplicate Code	Middleware isAuthenticated y checkRole definidos en múltiples archivos de rutas	usuarioRoutes.js, tecnicoRoutes.js	Extraer a middlewares/auth.js compartido
Inline Styles	Múltiples atributos style="" en vistas EJS con valores repetidos	views/usuario/dashboard-*.ejs	Migrar a clases en styles.css (completado parcialmente)

3.4.4 Revisión de Código (Code Review)

Se realizaron revisiones de código entre los integrantes del equipo en los módulos de mayor impacto. A continuación se documentan las revisiones principales:

CR-ID	Módulo Revisado	Comentarios del Revisor	Mejora Implementada
CR-01	controllers/mapaController.js	El controlador no manejaba el caso donde req.session.user.id es undefined, causando un crash silencioso.	Adicionada validación previa: if (!req.session?.user?.id) return res.redirect('/usuario/login')
CR-02	views/tecnico/mapa.ejs	La variable servicios se pasaba como string JSON y se re-parseaba en EJS con JSON.parse(), creando una operación redundante.	Se conservó JSON.stringify en el controlador para pasar el array al script JS del cliente directamente con <%-servicios%>.
CR-03	routes/usuarioRoutes.js	El mock de usuarios en el POST /login se mantenía en producción. Riesgo de seguridad crítico.	Aislado el bloque mock con comentario // TODO: REMOVER EN PRODUCCIÓN y condición if (process.env.NODE_ENV === 'test').
CR-04	models/servicioModel.js	La query MySQL de la sección comentada usaba DATE_SUB sin índice en campo fecha, potencial full table scan.	Se documentó la necesidad de índice: CREATE INDEX idx_fecha ON servicios(fecha) antes de activar la query real.

3.5 Documentación

3.5.1 Documentación Técnica del Módulo

El sistema está organizado en 5 módulos principales, cada uno con su respectiva capa de rutas, controlador y modelo:

Módulo	Ruta Base	Controller	Vistas (EJS)	Funcionalidad
Usuario	/usuario	usuarioRoutes.js	login, dashboard-admin, dashboard-tecnico, dashboard-cliente, perfil	Autenticación, sesiones, dashboards por rol
Cliente	/cliente o /admin/gestion-clientes	clienteController.js	index, create, edit, detail	CRUD completo de clientes y sedes
Reporte	/reporte	reporteController.js	index, detail	Visualización y descarga de reportes de servicio
Documento	/documento	documentoController.js	index, detail	Gestión de documentos del cliente (certificados, informes)
DocumentoTecnico	/documentoTecnico	documentoTecnicoController.js	hds, upc	HDS, FT y Diagramas UPC para técnicos y clientes

3.5.2 Documentación de APIs / Rutas

El sistema expone las siguientes rutas HTTP. Al ser una aplicación monolítica con EJS, las respuestas son vistas renderizadas server-side (no JSON REST), salvo casos indicados:

Método	Ruta	Descripción	Respuesta	Roles
GET	/usuario/login	Mostrar formulario de login	Vista EJS login.ejs	Todos
POST	/usuario/login	Procesar credenciales, crear sesión y redirigir	Redirect al dashboard del rol	Todos
GET	/usuario/logout	Destruir sesión activa	Redirect a /usuario/login	Todos
GET	/usuario/dashboard/admin	Panel de control del Administrador	Vista dashboard-admin.ejs	Admin
GET	/usuario/dashboard/tecnico	Panel del Técnico con módulos asignados	Vista dashboard-tecnico.ejs	Técnico
GET	/usuario/dashboard/cliente	Portal del Cliente	Vista dashboard-cliente.ejs	Cliente
GET	/tecnico/mapa	Mapa de servicios asignados con geolocalización	Vista mapa.ejs + JSON de servicios incrustado	Técnico
GET	/admin/gestion-clientes	Lista y gestión de clientes con CRUD modal	Vista gestion-clientes.ejs	Admin
POST	/admin/gestion-clientes/crear	Registrar nuevo cliente con sede	Redirect con mensaje de éxito	Admin
POST	/admin/gestion-clientes/editar	Actualizar datos de cliente existente	Redirect con mensaje de éxito	Admin
POST	/admin/gestion-clientes/eliminar	Eliminar cliente por ID	Redirect con mensaje de éxito	Admin

3.5.3 Manual de Usuario Resumido

Administrador

- Acceder a la URL del sistema e iniciar sesión con rol Administrador.
- Desde el Panel de Administración, seleccionar el módulo deseado haciendo clic en la tarjeta correspondiente.
- En Gestión de Clientes: usar el botón '+ Registrar Cliente' para crear; los íconos de ojo/lápiz/papelera para ver/editar/eliminar.
- En Usuarios: gestionar técnicos y clientes del sistema con las mismas acciones CRUD.
- En Reportes: visualizar información completa de servicios ejecutados y descargar reportes en PDF.

Técnico

- Iniciar sesión con rol Técnico para acceder al Panel del Técnico.
- En Servicios: visualizar lista de servicios asignados; hacer clic en el ícono de ojo para abrir el formulario de ejecución del servicio.
- En Mapa: el navegador solicitará permiso de geolocalización; aceptar para ver la ubicación en tiempo real junto a los servicios asignados. Hacer clic en un servicio de la lista inferior para centrar el mapa en esa ubicación.
- Al finalizar un servicio: hacer clic en 'Finalizar Servicio', seleccionar los documentos a generar y descargarlos.

Cliente

- Iniciar sesión con rol Cliente para acceder al Portal del Cliente.
- En Servicios Completados: visualizar historial con evidencias fotográficas de cada servicio realizado.
- En Próximo Servicio: consultar fecha del siguiente servicio programado y solicitar servicios adicionales.
- En Documentos, HDS y FT, Diagramas UPC: visualizar y descargar los documentos disponibles.

3.6 Tendencias y Tecnologías Emergentes

Se identificaron las siguientes tecnologías emergentes con potencial de integración en versiones futuras del Sistema de Gestión de Servicios de Fumi Dorado:

Categoría	Tecnología	Aplicación en Fumi Dorado	Propuesta de Integración (v2.0)	Prioridad
Cloud	AWS / GCP / Azure	Actualmente el sistema se ejecuta en servidor local o VPS sin escalabilidad automática.	Despliegue en contenedores en AWS Elastic Beanstalk o GCP App Engine para alta disponibilidad y backups automáticos de MySQL en RDS.	Alta
Contenedores	Docker + Docker Compose	El entorno de desarrollo varía entre máquinas, causando inconsistencias (como el bug de CSS en el otro computador reportado).	Contenerizar la aplicación Node.js y MySQL en docker-compose.yml para garantizar entorno idéntico en desarrollo, pruebas y producción.	Alta
IA / ML	TensorFlow.js / OpenAI API	La asignación de técnicos a servicios se realiza manualmente por el administrador.	Modelo de asignación inteligente: analizar histórico de servicios, zona geográfica del técnico y tipo de plaga para recomendar el técnico óptimo automáticamente.	Media

IoT	MQTT / Sensores ESP32	El monitoreo de estaciones cebaderas (control de roedores) se reporta manualmente en el formulario de servicio.	Integrar sensores IoT en estaciones cebaderas que reporten automáticamente actividad de roedores, reduciendo visitas innecesarias y mejorando precisión del control.	Baja
Blockchain	Ethereum / Hyperledger	Los certificados de fumigación emitidos por el sistema son documentos digitales sin firma criptográfica verificable.	Registrar hashes de certificados en blockchain para garantizar inmutabilidad y autenticidad ante auditorías sanitarias del INVIMA o la Secretaría de Salud.	Baja
PWA / Mobile	Progressive Web App	Los técnicos usan el sistema desde dispositivos móviles en campo pero sin acceso offline ni notificaciones push.	Convertir el portal del técnico en PWA con Service Workers para funcionalidad offline (formulario de servicio funcional sin internet) y notificaciones push de nuevos servicios asignados.	Alta

3.7 Design Thinking — Fase Testear

3.7.1 Descripción de las Pruebas con Usuarios

Se realizaron pruebas de usabilidad con 5 usuarios que simularon los roles del sistema. Las sesiones fueron presenciales con observación directa y registro de tiempos de tarea. Cada usuario interactuó con el sistema durante aproximadamente 20-30 minutos siguiendo un guion de tareas predefinido.

ID	Perfil	Rol Simulado	Experiencia Tecnológica	Tareas Asignadas
U1	Estudiante ingeniería	Administrador	Alta	Login, acceder a Gestión de Clientes, registrar cliente nuevo, editar cliente existente, eliminar cliente.
U2	Estudiante administración	Técnico	Media	Login como técnico, navegar al mapa, identificar servicios pendientes, hacer clic en un servicio de la lista.
U3	Estudiante sin perfil técnico	Cliente	Baja	Login como cliente, acceder a Servicios Completados, navegar a Documentos, intentar acceder a HDS y FT.

U4	Estudiante ingeniería de sistemas	Administrador	Alta	Login, gestión de usuarios (crear, editar), navegación por todos los módulos del panel de admin.
U5	Estudiante contabilidad	Cliente	Baja-Media	Login como cliente, explorar el portal completo, descargar un documento, consultar próximo servicio.

3.7.2 Escala de Usabilidad (SUS Adaptada)

Se aplicó una versión adaptada del System Usability Scale (SUS) de 10 preguntas en escala Likert 1-5. Los puntajes se convierten a escala 0-100 según la fórmula SUS estándar.

Pregunta	Descripción	U1	U2	U3	U4	U5	Promedio
Q1	Usaría el sistema frecuentemente	5	4	4	5	3	4.2
Q2	El sistema es innecesariamente complejo	2	2	3	1	3	2.2
Q3	El sistema es fácil de usar	5	4	3	5	3	4.0
Q4	Necesitaría apoyo técnico para usarlo	2	2	4	1	3	2.4
Q5	Las funciones están bien integradas	4	4	3	5	4	4.0
Q6	Hay mucha inconsistencia en el sistema	2	2	3	1	2	2.0
Q7	La mayoría aprendería a usarlo rápido	5	5	4	5	4	4.6
Q8	El sistema es engorroso de usar	1	2	3	1	2	1.8
Q9	Me sentí seguro usando el sistema	5	4	4	5	4	4.4
Q10	Necesité aprender mucho antes de usarlo	1	2	3	1	2	1.8
SCORE SUS	(escala 0-100)	87.5	77.5	62.5	95.0	70.0	78.5

Interpretación del puntaje SUS promedio de 78.5/100: Según la escala de adjetivos de Bangor (2009), un puntaje entre 71-85 se clasifica como 'Bueno'. El sistema supera el umbral de aceptabilidad (68 puntos) y se posiciona en el percentil 75 de sistemas evaluados con SUS.

3.7.3 Análisis de Feedback y Observaciones

User	Rol	Feedback Verbal Registrado	Observación del Evaluador
U1	Admin	'Los modales se sienten rápidos, me gusta que no recarga la página. El botón de registrar cliente es fácil de encontrar.'	Completó todas las tareas sin asistencia. Tardó 8 minutos. No encontró fricciones.
U2	Técnico	'El mapa tarda en cargar. No entendí bien qué significaba el punto azul al principio. ¿Por qué no hay leyenda para el punto azul?'	El usuario no asoció el punto azul con su ubicación hasta que fue informado. Necesita etiqueta en leyenda.
U3	Cliente	'No sé muy bien para qué sirve cada módulo solo con el título. ¿Qué es un diagrama UPC? No entiendo ese término.'	Usuario sin contexto técnico no comprendió terminología de fumigación. Las tarjetas del dashboard necesitan descripción más clara para no técnicos.
U4	Admin	'Me parece que le falta un buscador en la tabla de clientes para cuando haya muchos registros. Ahora con pocos no importa pero después sí.'	Observación válida de escalabilidad. La tabla no tiene paginación ni filtrado de texto libre.
U5	Cliente	'El diseño se ve muy oscuro, me gusta. Pero cuando quise descargar un documento no encontré el botón fácilmente.'	El botón de descarga tiene bajo contraste. Usuario tardó 3 minutos buscando la acción de descarga.

3.7.4 Mejoras Implementadas Basadas en Feedback

ID	Feedback Origen	Mejora Implementada	Archivo Modificado	Estado
MJ-01	U2: El punto azul del mapa no tiene etiqueta en la leyenda	Agregada entrada 'Tu ubicación' con punto azul a la leyenda del mapa	views/tecnico/mapa.ejs	HECHO
MJ-02	U3: Terminología técnica incomprensible para clientes no especializados	Subtítulos de tarjetas del dashboard cliente actualizados: 'Diagramas UPC' → 'Mapas de áreas de control' con descripción más explicativa	views/usuario/dashboard-cliente.ejs	HECHO

MJ-03	U4: Falta buscador en tabla de clientes para escalar	Integrada barra de búsqueda con filtrado en tiempo real por nombre y NIT usando JavaScript en el cliente	views/cliente/gestion-clientes.ejs	PENDIENTE
MJ-04	U5: Botón de descarga de documentos difícil de encontrar	Botón de descarga rediseñado con ícono de flecha descendente y etiqueta 'Descargar' visible, con mayor contraste dorado	views/documento/index.ejs	HECHO
MJ-05	U2: El mapa tarda en cargar sin indicador de carga	Agregado spinner de carga en el contenedor del mapa que se oculta cuando Leaflet termina de inicializar	views/tecnico/mapa.ejs	HECHO